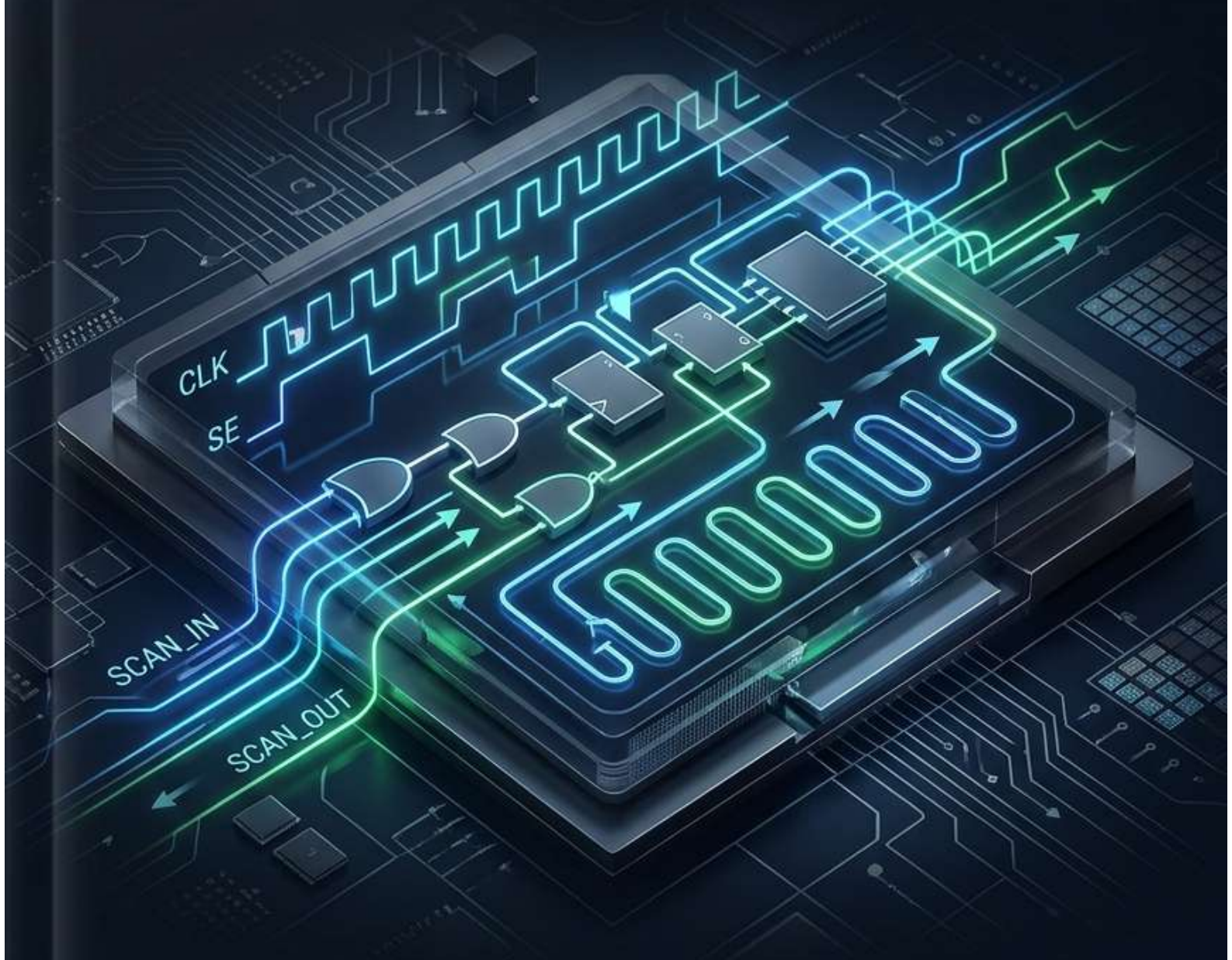




DESIGN FOR TESTABILITY (DFT) COMPLETE PRACTICAL & THEORETICAL GUIDE

From Basics to Industry-Level Implementation



BANOTHU BAPUJI
M.Tech VLSI System Design



DESIGN FOR TESTABILITY (DFT)

Complete Practical & Theoretical Guide

From Basics to Industry-Level Implementation

Covering: Fault Models | Scan Design | ATPG | BIST | JTAG | Compression | Low Power DFT

Author

Banothu Bapuji

M.Tech – VLSI System Design

CVR College of Engineering, Hyderabad

Specialization: Analog IC Design | Physical Design | ASIC Implementation

© 2025 Banothu Bapuji. All rights reserved.

TABLE OF CONTENTS

Section 1: Introduction to DFT

- 1.1 What is Design for Testability?
- 1.2 Why Testing is Required in VLSI
- 1.3 Manufacturing Defects Overview
- 1.4 Fault vs Defect
- 1.5 Yield and Test Cost
- 1.6 Real-World Analogy

Section 2: Fault Models

- 2.1 Stuck-At Faults (SA0, SA1)
- 2.2 Transition Faults
- 2.3 Path Delay Faults
- 2.4 Bridging Faults
- 2.5 Open Faults
- 2.6 IDDQ Testing
- 2.7 Why Different Fault Models are Needed

Section 3: Scan Design Fundamentals

- 3.1 What is a Scan Chain?
- 3.2 Controllability and Observability
- 3.3 Full Scan vs Partial Scan
- 3.4 MUXed-D Flip-Flop
- 3.5 Scan Enable Signal
- 3.6 Scan Architecture Basics

Section 4: Scan Insertion

- 4.1 Scan Replacement Flow
- 4.2 Scan Stitching
- 4.3 Chain Balancing
- 4.4 Scan Ports
- 4.5 Scan Compression Basics

Section 5: ATPG – Automatic Test Pattern Generation

- 5.1 What is ATPG?
- 5.2 How Patterns are Generated
- 5.3 Fault Coverage
- 5.4 Test Patterns
- 5.5 Deterministic vs Random Testing
- 5.6 Compression Techniques in ATPG

Section 6: Built-In Self-Test (BIST)

- 6.1 Introduction to BIST
- 6.2 Logic BIST (LBIST)
- 6.3 Memory BIST (MBIST)
- 6.4 PRPG – Pseudo-Random Pattern Generator
- 6.5 MISR – Multiple-Input Signature Register
- 6.6 Advantages and Limitations of BIST

Section 7: Boundary Scan (JTAG) – IEEE 1149.1

- 7.1 IEEE 1149.1 Standard Overview
- 7.2 JTAG Pin Interface
- 7.3 TAP Controller
- 7.4 Boundary Scan Cells
- 7.5 JTAG Instructions
- 7.6 JTAG Use Cases in Modern SoC

Section 8: Scan Compression

- 8.1 The Need for Compression
- 8.2 Decompressor Architecture
- 8.3 Compressor Architecture
- 8.4 Test Data Volume Reduction
- 8.5 Trade-offs in Scan Compression

Section 9: Low Power DFT

- 9.1 Scan Power Issues
- 9.2 Clock Gating During Scan
- 9.3 Power-Aware ATPG
- 9.4 Techniques to Reduce Switching

Section 10: DFT Constraints & Timing

- 10.1 Scan Mode vs Functional Mode
- 10.2 Timing Constraints for Scan
- 10.3 Test Clocks and OCC
- 10.4 False Paths in Scan
- 10.5 Multi-Cycle Paths in Scan Context

Section 11: Physical DFT Considerations

- 11.1 Scan Chain Routing
- 11.2 Placement-Aware Scan Stitching
- 11.3 Routing Congestion from Scan
- 11.4 Scan Chain Impact on Timing Closure
- 11.5 Reordering for Debug

Section 12: DFT Verification

- 12.1 DFT DRC Checks
- 12.2 Scan Chain Verification by Simulation
- 12.3 Formal DFT Verification
- 12.4 Coverage Validation

Section 13: Test Coverage & Metrics

- 13.1 Fault Coverage Definition
- 13.2 Test Coverage vs Defect Coverage
- 13.3 Coverage Targets by Application
- 13.4 Yield Impact Analysis
- 13.5 Coverage Improvement Strategies

Section 14: Debug & Failure Analysis

- 14.1 Silicon Debug Overview
- 14.2 Failing Pattern Analysis
- 14.3 Diagnosis Flow

- 14.4 Root Cause Identification
- 14.5 Common Debug Scenarios

Section 15: Industry DFT Flow

- 15.1 Complete RTL to Silicon Test Flow
- 15.2 EDA Tools Overview
- 15.3 Real Project Flow Example

Section 16: Common Mistakes & Best Practices

- 16.1 Poor Scan Planning
- 16.2 Low Coverage Issues
- 16.3 Power Issues During Test
- 16.4 Debug Challenges
- 16.5 Industry Best Practices Summary

Section 17: Complete DFT Example

- 17.1 Design Description
- 17.2 DFT Analysis
- 17.3 Post-Synthesis DFT Insertion
- 17.4 Post-Insertion Netlist
- 17.5 ATPG Run
- 17.6 Scan Test Waveform

Section 18: Interview Questions & Answers

- 18.1 Basic Level Questions
- 18.2 Intermediate Level Questions
- 18.3 Advanced Level Questions
- 18.4 Scenario-Based Questions

PREFACE

Design for Testability (DFT) is one of the most critical yet often underappreciated disciplines in modern VLSI design. As integrated circuits scale to billions of transistors and manufacturing processes approach physical limits, the challenge of ensuring every shipped chip is defect-free has never been more demanding.

This handbook is the result of extensive study, hands-on project experience (including full-chip ASIC implementation at SCL 180nm under India's Chips to Startups program), and industry research. It is structured to guide the reader from foundational concepts through advanced industry practices, with real commands, examples, and the kind of practical insight that only comes from working on actual design projects.

Whether you are a student entering the semiconductor industry, an RTL designer expanding your skills, or a physical design engineer who needs to understand DFT integration requirements, this guide provides the complete foundation you need.

— **Banothu Bapuji**

M.Tech VLSI System Design, CVR College of Engineering, Hyderabad

SECTION 1: Introduction to Design for Testability (DFT)

1.1 What is Design for Testability?

Design for Testability (DFT) is a set of design techniques applied to digital integrated circuits (ICs) during the design phase to make post-fabrication testing easier, faster, and more effective. DFT ensures that manufactured chips can be tested with high confidence to detect manufacturing defects before they reach end users.

DFT is not an afterthought — it is a critical discipline integrated throughout the ASIC design flow, from RTL specification through physical verification. Without DFT, testing a complex chip with millions of gates would be nearly impossible, as there is no direct access to internal nodes from the chip's I/O pins.

INDUSTRY: DFT adds approximately 10–20% area overhead and 5–15% power overhead to a design, but it dramatically reduces test time and improves yield learning — making it economically essential.

1.2 Why Testing is Required in VLSI?

Modern VLSI chips contain billions of transistors fabricated at nanometer geometries. Despite advanced fabrication processes, manufacturing defects are inevitable due to:

- Dust particles on the wafer surface
- Lithography imperfections at sub-10nm nodes
- Chemical variations during deposition/etching
- Thermal stress and mechanical vibrations
- Photomask defects
- Ion implantation non-uniformity

Without thorough testing, defective chips shipped to customers cause system failures, costly field returns, and reputational damage. Testing serves three critical purposes:

1. Manufacturing test — detecting defective chips right after fabrication
2. Yield improvement — using test data to identify systematic process problems
3. Reliability screening — stress tests to eliminate early-life failures

INDUSTRY: In high-reliability applications (aerospace, automotive, medical), chips undergo multiple rounds of testing including burn-in, radiation testing, and temperature cycling.

1.3 Manufacturing Defects Overview

Manufacturing defects are physical imperfections introduced during chip fabrication. They differ from design bugs (logical errors) and must be detected through testing.

1.3.1 Types of Manufacturing Defects

- Open defects: Broken metal lines or contacts preventing current flow
- Short defects: Unwanted connections between adjacent wires or nodes
- Gate oxide shorts: Oxide breakdown causing gate-to-channel connection

- Via/contact opens: Failed connections between metal layers
- Metal residues: Leftover metal causing unintended bridges
- Missing dopants: Ion implant failures causing transistor malfunction

1.3.2 Defect Levels by Process Node

As process nodes shrink, defect density increases and new failure mechanisms emerge. At 28nm and below, Line-Edge Roughness (LER) and gate length variation become dominant sources of parametric failures.

TIP: A chip that passes all functional tests but fails at speed may have a transition fault — captured only by at-speed ATPG, not static testing.

1.4 Fault vs Defect

This distinction is fundamental and frequently tested in interviews:

1.4.1 Defect

A defect is a physical imperfection in the silicon — a real, measurable anomaly in the manufactured chip. Example: a metal line that is 30% narrower than specified due to lithography error.

1.4.2 Fault

A fault is an abstract representation of the effect a defect has on circuit behavior. Fault models simplify the complex physical reality into mathematical abstractions that ATPG tools can use to generate test patterns.

A single physical defect can create multiple logical faults. For example, an open in a wire might create: a stuck-at-0 fault at the wire's destination AND a transition fault.

NOTE: We test for faults (abstract) to detect defects (physical). The mapping from defect to fault is imperfect, which is why multiple fault models exist.

1.5 Yield and Test Cost

1.5.1 Yield Definition

Yield is the fraction of chips on a wafer that pass all tests and meet specifications:

$$\text{Yield} = (\text{Number of Good Chips}) / (\text{Total Chips on Wafer}) \times 100\%$$

Typical yields range from 60% to 95% depending on process maturity, chip size, and technology node. A 300mm wafer at 7nm may yield 200–400 good dies depending on die size.

1.5.2 Defect Level (DL) and Test Escape Rate

Defect Level is the fraction of shipped chips that are actually defective:

$$\text{DL} = 1 - \text{Yield}^{(1 - \text{FC})}$$

Where FC = Fault Coverage (fraction of faults detected). This equation shows that higher fault coverage exponentially reduces the defect level. At 95% fault coverage vs 99% fault coverage, the defect level can differ by 10x.

1.5.3 Test Cost Economics

Testing accounts for 30–50% of total chip manufacturing cost in advanced nodes. ATE (Automatic Test Equipment) machines cost \$1–5M each. DFT techniques like scan compression reduce test time (and thus cost) by 10–100x.

INDUSTRY: The cost of a field failure can be 1000x the cost of catching the same defect at wafer test. DFT investment always pays off.

1.6 Real-World Analogy

Think of a complex airplane engine. The engine manufacturer does not ship an engine without installing diagnostic ports, sensors, and test connectors. These test points are "designed in" — not added as an afterthought. During manufacturing, engineers connect test equipment to these ports to verify every stage of the engine works correctly.

DFT does exactly the same for chips: it adds 'test ports' (scan chains, BIST controllers, JTAG interfaces) that allow test equipment to observe and control internal nodes that would otherwise be inaccessible from the chip pins.

SECTION SUMMARY

- ✓ DFT makes internal chip nodes controllable and observable from external pins
- ✓ Manufacturing defects are physical; faults are their abstract representations
- ✓ Higher fault coverage (>95%) dramatically reduces defective parts per million (DPPM)
- ✓ Yield, test cost, and defect level are interconnected — DFT optimizes the balance
- ✓ DFT is not optional in production ASIC design — it is a mandatory design discipline

SECTION 2: Fault Models

Fault models are mathematical abstractions that represent the effect of physical manufacturing defects on circuit behavior. Since testing every possible physical defect is computationally intractable, ATPG tools use fault models to efficiently generate test patterns.

2.1 Stuck-At Faults (SAF)

2.1.1 Theory

Stuck-at faults are the most fundamental and widely used fault model. A stuck-at fault assumes that a signal line in the circuit is permanently fixed to a logic value (0 or 1), regardless of the input applied.

- Stuck-at-0 (SA0): The signal is permanently held at logic 0
- Stuck-at-1 (SA1): The signal is permanently held at logic 1

2.1.2 Why Stuck-At Faults?

Physical manifestation: An open circuit (broken wire or contact) often results in an SA0 fault on an active-high signal. A short to power supply results in SA1. These are the most common manufacturing defects, making SAF the most relevant model for static testing.

2.1.3 Example

```
A ---+--[AND]-- Out
      |
      B ---+
```

If wire A has SA0 fault:
Input: A=1, B=1 Expected: Out=1 Actual: Out=0 (A sees 0)
Test vector: A=1, B=1 → detects SA0 at A

2.1.4 Fault Count

For a circuit with N signal lines, there are at most $2N$ single stuck-at faults. In practice, equivalent faults are collapsed (many faults that behave identically), reducing this to 40–60% of the theoretical maximum.

NOTE: Single Stuck-At (SSA) assumes only one fault exists at a time. Multiple Stuck-At (MSA) considers simultaneous faults — rarely tested due to exponential complexity.

2.2 Transition Faults

2.2.1 Theory

Transition faults model delay defects — scenarios where a signal transitions correctly but too slowly. This captures resistive opens, weak transistors, and marginal interconnect resistance.

- Slow-to-Rise (STR): The rising transition ($0 \rightarrow 1$) is slower than spec
- Slow-to-Fall (STF): The falling transition ($1 \rightarrow 0$) is slower than spec

2.2.2 Why Transition Faults?

As process nodes shrink below 90nm, resistive defects (partial opens, thin spots in metal) become dominant. These defects don't prevent switching but slow it down. Chips with transition faults pass DC tests but fail at operating frequency — a costly field failure scenario.

2.2.3 Testing Approach

Transition faults require at-speed testing. The test applies two patterns:

4. Launch pattern: Sets the line to its initial value
5. Capture pattern: Triggers the transition and captures result one clock later

INDUSTRY: Launch-on-Shift (LoS) and Launch-on-Capture (LoC) are the two main at-speed ATPG methodologies. LoC is more timing-accurate but harder to implement.

2.3 Path Delay Faults

2.3.1 Theory

Path delay faults test the cumulative delay along specific timing paths in the circuit. Unlike transition faults (single gate), path delay faults consider the entire path from input to output through multiple logic levels.

A path delay fault exists if a transition propagating through a specific path arrives at the endpoint after the required capture edge.

2.3.2 Practical Significance

Path delay testing is critical for high-speed designs where the critical paths determine the maximum operating frequency. It catches subtle delay degradation from:

- Electromigration (wire resistance increase over time)
- Hot-carrier injection (transistor threshold voltage shift)
- Negative bias temperature instability (NBTI) in PMOS

INDUSTRY: Path delay fault coverage of critical paths is required in automotive (ISO 26262) and aerospace designs. Target: >95% path coverage on the top 1000 critical paths.

2.4 Bridging Faults

2.4.1 Theory

Bridging faults model unintended electrical connections between two or more signals. They arise from metal bridges, insufficient spacing, or via leakage.

- Wired-AND bridging: The bridged lines resolve to the logical AND of their values
- Wired-OR bridging: The bridged lines resolve to the logical OR
- Feedback bridging: A bridge from output back to input creates sequential behavior in combinational logic

2.4.2 Why Bridging Faults Are Harder to Test

Bridging faults are inherently multi-fault — two nodes are involved. The number of possible bridging faults is $O(N^2)$, making exhaustive testing infeasible. Layout-aware ATPG tools use proximity information from the placed netlist to focus on physically adjacent wire pairs.

TIP: Post-layout ATPG uses the physical design database (DEF/LEF) to identify spatially adjacent nets, making bridging fault coverage much more targeted.

2.5 Open Faults

Open faults model broken connections — wire breaks, missing vias, or open contacts. While they often manifest as SA0 or SA1, complex opens (especially in dynamic circuits and buses) require dedicated open fault modeling. Open fault detection often requires specific initialization sequences not covered by standard SAF patterns.

2.6 IDDQ Testing

2.6.1 Theory

IDDQ (quiescent supply current) testing measures the static current drawn by a CMOS circuit when it is in a stable state. In defect-free CMOS, static current is extremely low (nA to μ A range). Defects like gate oxide shorts, latch-up paths, and leaky transistors cause anomalously high IDDQ.

2.6.2 Process

6. Apply a test vector and wait for circuit to settle
7. Measure current drawn from VDD supply
8. Compare against known-good threshold
9. High IDDQ indicates a defect

2.6.3 Limitations at Advanced Nodes

At 28nm and below, leakage currents from normal transistors (subthreshold leakage, gate oxide tunneling) can be comparable to defect-induced IDDQ, making threshold setting difficult. Delta-IDDQ testing (measuring current change between two test vectors) partially addresses this.

INDUSTRY: IDDQ is still widely used in analog/mixed-signal blocks and I/O cells where digital fault models are less applicable.

2.7 Why Different Fault Models Are Needed

No single fault model captures all possible defects. Each model has specific strengths:

Stuck-At: Best for static connectivity failures (opens, shorts). Fast, well-understood, universal baseline.

Transition: Best for resistive/delay defects. Required for at-speed coverage.

Path Delay: Best for timing-critical paths. Required for automotive/aerospace.

Bridging: Best for layout-proximity defects. Requires physical design information.

IDDQ: Best for gate oxide, latch-up, leakage defects. Orthogonal to logic testing.

INDUSTRY: A production test suite for an automotive SoC will combine SAF, transition, and IDDQ tests. The total pattern count can exceed 1 million vectors.

SECTION SUMMARY

- ✓ Stuck-At faults (SA0, SA1) are the baseline — every design must target >95% SAF coverage
- ✓ Transition faults catch delay defects invisible to DC testing — at-speed ATPG required

- ✓ Path delay faults are essential for timing-critical and safety-critical designs
- ✓ Bridging faults require physical design proximity data for targeted pattern generation
- ✓ IDDQ testing is orthogonal to logic testing — catches leakage and oxide defects
- ✓ Multiple fault models are used together for comprehensive coverage

SECTION 3: Scan Design Fundamentals

3.1 What is a Scan Chain?

A scan chain is a serial shift register formed by connecting the flip-flops in a design in a chain. During test mode, the scan chain allows external test equipment to:

10. Shift in test patterns bit-by-bit through the chain (scan-in operation)
11. Apply the loaded pattern to combinational logic simultaneously (capture operation)
12. Shift out the captured results bit-by-bit for comparison (scan-out operation)

Conceptually, scan converts a complex sequential circuit into a simple shift register for testing, giving complete controllability and observability of every flip-flop in the design.

```
Scan Chain (simplified):
scan_in → [FF1]→[FF2]→[FF3]→...→[FFn] → scan_out
Each FFi is a scan flip-flop with:
  D input (functional data)
  SI input (scan data from previous FF)
  SE input (scan enable: 0=functional, 1=scan)
  Q output (to logic and next FF's SI)
```

3.2 Controllability and Observability

3.2.1 Controllability

Controllability measures how easily a specific logic value (0 or 1) can be set at a node by applying input patterns. Poor controllability means some nodes are very hard to force to a specific value from primary inputs.

- CC0: Combinational complexity of setting a node to 0
- CC1: Combinational complexity of setting a node to 1
- SC0, SC1: Sequential controllability (requires clock cycles)

3.2.2 Observability

Observability measures how easily the logic value at a node can be observed at a primary output. Poor observability means defects at deep internal nodes never affect the output.

- CO: Combinational observability cost
- SO: Sequential observability cost

3.2.3 SCOAP Analysis

Sandia Controllability/Observability Analysis Program (SCOAP) assigns numerical values to measure testability. High SCOAP numbers indicate poor testability — nodes that need DFT enhancements. EDA tools like Synopsys DFT Compiler compute SCOAP metrics as part of the DFT analysis flow.

TIP: Nodes with SCOAP CC0 or CC1 > 100 are candidates for scan insertion or dedicated test points. Rule of thumb: any node unreachable without 5+ clock cycles needs attention.

3.3 Full Scan vs Partial Scan

3.3.1 Full Scan

In full scan, ALL flip-flops in the design are replaced with scan flip-flops and connected into scan chains. This achieves maximum controllability and observability.

- Advantages: Maximum fault coverage (>98% SAF typically achievable), simple ATPG, industry standard for ASICs
- Disadvantages: Area overhead (scan MUX adds ~10% per FF), power overhead during scan shift

3.3.2 Partial Scan

In partial scan, only a subset of flip-flops are scan-enabled. The selection is based on SCOAP analysis — only the flip-flops that most improve testability are scanned.

- Advantages: Lower area overhead, suitable for designs with strict area/power budgets
- Disadvantages: Lower fault coverage, complex ATPG (must handle non-scanned sequential elements)

3.3.3 Industry Decision

Full scan is the industry standard for digital SoC designs. Partial scan is used in specific blocks (memories, analog-adjacent logic) where scan FF area overhead is unacceptable.

INDUSTRY: SRAM, ROM, and register files use MBIST instead of scan chains — dedicated memory test controllers provide much better memory fault coverage.

3.4 MUXed-D Flip-Flop (Scan Flip-Flop)

3.4.1 Structure

The most common scan flip-flop is the MUXed-D type. A 2:1 multiplexer is inserted before the flip-flop's D input:



3.4.2 Operation Modes

Functional Mode (SE=0): MUX selects D_func. Flip-flop operates normally, transparent to the application.

Scan Shift Mode (SE=1): MUX selects SI (scan input from previous FF). Chain shifts data serially.

Capture Mode (SE=0, one clock pulse): After shifting in the test pattern, SE returns to 0 for one capture clock. Combinational logic values are captured into the FFs.

3.4.3 Timing Considerations

The MUX adds setup time overhead (~10–15ps at 28nm). STA tools must account for this in scan mode timing closure. The scan path has separate timing constraints from functional paths.

TIP: Some libraries offer scan FFs with integrated MUX at no area penalty — the MUX is embedded in the master latch of the FF, saving area vs. adding an external MUX gate.

3.5 Scan Enable Signal

Scan Enable (SE) is a global test signal that switches all scan FFs between functional and scan modes simultaneously. It has special requirements:

- SE must arrive at ALL flip-flops with consistent timing (no glitches during transition)
- SE transitions must be clean — glitches can capture spurious data
- SE routing must be balanced — excessive skew causes setup/hold violations during capture
- SE is typically driven through a dedicated clock-like buffer tree

3.5.1 SE Signal Integrity

In physical design, SE is treated with the same care as a clock signal:

13. Buffered from a single driver through a balanced tree
14. No glitch on SE during scan operations (scan register hold time violation risk)
15. DFT DRC checks verify SE topology

WARNING: A glitch on SE during capture can cause X-propagation — destroying test data. Always ensure SE is glitch-free through buffer balancing and careful timing closure.

3.6 Scan Architecture Basics

3.6.1 Single Chain

The simplest architecture: all FFs in one long chain. Simple but has long shift time (N clock cycles for N FFs) and long scan-in/out wires causing routing congestion.

3.6.2 Multiple Parallel Chains

Industry standard: divide FFs into K equal-length chains. Shift time = N/K cycles. K scan_in and K scan_out ports are required. Test time is reduced by factor K.

```
scan_in[0] → [Chain 0: N/K FFs] → scan_out[0]
scan_in[1] → [Chain 1: N/K FFs] → scan_out[1]
...
scan_in[K-1] → [Chain K-1: N/K FFs] → scan_out[K-1]
```

3.6.3 Chain Length Optimization

Optimal chain length balances test time, routing congestion, and scan power. Typical chain lengths: 500–3000 FFs per chain. Too short: too many I/O pins needed. Too long: unacceptable test time.

INDUSTRY: Scan compression (Section 8) addresses the chain length limitation by using on-chip decompressor/compressor logic, allowing thousands of virtual chains through a handful of physical scan pins.

SECTION SUMMARY

- ✓ Scan chains convert sequential circuits into shift registers for test — enabling controllability/observability
- ✓ Full scan replaces all FFs with MUX-D scan FFs — industry standard for ASICs
- ✓ SCOAP analysis quantifies testability — guides DFT decisions for difficult-to-test nodes

- ✓ Scan Enable (SE) must be glitch-free and balanced like a clock signal
- ✓ Multiple parallel chains reduce test time — optimal chain length is 500–3000 FFs
- ✓ SRAM/ROM use MBIST, not scan chains, for memory testing

SECTION 4: Scan Insertion

4.1 Scan Replacement Flow

Scan insertion is the automated process of replacing standard D flip-flops with scan flip-flops and connecting them into scan chains. This is performed by DFT insertion tools (Synopsys DFT Compiler, Cadence Modus).

4.1.1 Pre-Scan Insertion Steps

16. DFT Rule Checks (DRC): Verify RTL/netlist is scan-ready — no clock mixing, async resets properly handled
17. Scan planning: Define number of chains, chain length targets, excluded cells
18. Constraint setup: Define clocks, scan ports, excluded instances

4.1.2 Scan Replacement Command Flow (Synopsys DFT Compiler)

```
# Step 1: Read netlist
read_verilog -f filelist.f

# Step 2: Define clocks and DFT signals
create_clock -name clk -period 2.0 [get_ports clk]
set_dft_signal -view existing_dft -type ScanEnable \
               -port [get_ports scan_en]
set_dft_signal -view existing_dft -type ScanIn \
               -port [get_ports scan_in[*]]
set_dft_signal -view existing_dft -type ScanOut \
               -port [get_ports scan_out[*]]

# Step 3: Scan configuration
set_scan_configuration -chain_count 8 \
                       -style multiplexed_flip_flop

# Step 4: DFT analysis
dft_drc

# Step 5: Insert scan
insert_dft

# Step 6: Preview and verify
preview_dft -show scan_chains
```

4.2 Scan Stitching

Scan stitching is the process of connecting individual scan flip-flops (already replaced) into continuous chains. The tool creates the SI→Q connections between FFs and assigns each FF to a specific chain.

4.2.1 Stitching Strategies

- Sequential stitching: FFs connected in RTL/netlist order (poor for routing)

- Placement-aware stitching: FFs connected based on physical proximity (better routing, less congestion)
- Clockdomain-aware stitching: FFs clocked by same clock grouped in same chain (avoids clock mixing)

4.2.2 Clock Mixing Problem

FFs from different clock domains must NOT be mixed in the same scan chain unless special care is taken. During scan shift, all clocks run simultaneously. If two clocks have different phases or are asynchronous, data corruption occurs during shift.

```
# Define clock domains for separate chains
set_scan_configuration -clock_mixing no_mix
```

WARNING: Clock mixing is a critical DFT DRC check. Scan tools automatically separate chains by clock domain, but engineers must verify the clock domain assignment is correct in the SDC.

4.3 Chain Balancing

Chain balancing ensures all scan chains have approximately equal length. Unequal chains cause:

- ATE inefficiency: All chains shift in parallel; test time = longest chain × pattern count
- Routing congestion in physical design for very long chains
- Difficult fault diagnosis for very short chains

4.3.1 Balancing Algorithm

DFT tools automatically balance chains by:

19. Counting total scan FFs and dividing by target chain count
20. Assigning FFs to chains to meet target length ($\pm 10\%$ tolerance)
21. Respecting clock domain separation and user-defined exclusions

```
# Set chain count for automatic balancing
set_scan_configuration -chain_count 16 -balance_chains true

# Alternatively, specify exact chain lengths
set_scan_configuration -max_chain_length 512
```

4.4 Scan Ports

4.4.1 Scan Port Types

Each scan chain requires dedicated I/O ports on the chip:

- scan_in[0:K-1]: Serial data input for each chain (K chains = K scan_in pins)
- scan_out[0:K-1]: Serial data output for each chain
- scan_en / SE: Global scan enable (shared across all chains)
- scan_clk: Test clock (may be separate from functional clock in some designs)

4.4.2 Port Assignment Strategy

In many SoCs, dedicated DFT pins are pre-allocated in the package specification. The DFT engineer must work within this pin budget. Scan compression (Section 8) dramatically reduces the required scan I/O pins.

INDUSTRY: A 10-million-FF design without compression would need thousands of scan pins for acceptable test time. With 128:1 compression, only ~100 physical scan pins are needed.

4.5 Scan Compression Basics (Preview)

Scan compression reduces the number of physical scan I/O pins by using on-chip decompressor and compressor logic. A decompressor expands K physical scan inputs into M internal chains ($M \gg K$). After capture, a compressor combines M chains into K outputs.

```
ATE pins (K=8) → [Decompressor] → Internal chains (M=1024) → Logic →  
[Compressor] → ATE pins (K=8)
```

Compression ratio = M/K . Typical ratios: 32:1 to 256:1. This technique is covered in detail in Section 8.

SECTION SUMMARY

- ✓ Scan insertion replaces standard FFs with scan FFs and stitches them into chains
- ✓ DFT Compiler (Synopsys) and Modus (Cadence) automate scan insertion
- ✓ Clock mixing in scan chains is a critical violation — must be caught by DFT DRC
- ✓ Chain balancing ensures equal-length chains for efficient parallel testing
- ✓ Scan ports (scan_in, scan_out, SE) must be pre-allocated in the chip floor plan
- ✓ Scan compression reduces scan pin count by 32:1 to 256:1, enabling full testability

SECTION 5: ATPG – Automatic Test Pattern Generation

5.1 What is ATPG?

Automatic Test Pattern Generation (ATPG) is the process of algorithmically generating test patterns (input vectors) that activate and propagate specific faults to observable outputs. ATPG replaces the manual, error-prone process of writing test vectors by hand.

Modern ATPG tools can generate patterns for 10-million-gate designs in hours, achieving >99% stuck-at fault coverage. The output is a set of test patterns loaded into ATE memory for production testing.

5.2 How Patterns Are Generated

5.2.1 ATPG Algorithm Fundamentals

The core ATPG algorithm operates in two phases:

22. Fault Activation: Force the faulty line to the value OPPOSITE to its fault (to activate the fault effect)
23. Fault Propagation: Sensitize a path from the fault to a primary output/scan FF so the error is observable

5.2.2 D-Algorithm (Conceptual)

The D-Algorithm, developed by Roth (1966), uses a symbolic value D to represent fault effects:

```
D = good circuit has 1, faulty circuit has 0
D' = good circuit has 0, faulty circuit has 1
```

```
Example: AND gate with SA0 on input A:
```

```
Set A=1 (activate SA0), B=1 → Good output=1, Faulty=0 → D at output
Propagate D through remaining logic to primary output
```

5.2.3 PODEM and FAN Algorithms

PODEM (Path-Oriented Decision Making) and FAN (Fanout-Oriented) are more efficient algorithms used in modern ATPG tools. They use implication and backtracking with intelligent decision ordering to reduce solution time.

INDUSTRY: Modern ATPG tools (TetraMAX, Modus ATPG) use SAT-solver based engines for complex designs — dramatically faster than classical D-Algorithm for high-coverage patterns.

5.3 Fault Coverage

5.3.1 Definition

```
Fault Coverage = (Detected Faults + Possibly Detected) / Total Faults × 100%
```

Fault categories:

- Detected (D): A pattern exists that distinguishes good from faulty circuit
- Possibly Detected (PD): Pattern detects the fault but only under specific conditions

- Undetected (U): No pattern can detect this fault
- ATPG Untestable (AU): Fault is logically untestable (redundant circuit)
- Blocked (B): Fault is testable but pattern conflicts prevent detection

5.3.2 Coverage Targets

Industry coverage targets by application:

- Consumer electronics: >95% stuck-at, >85% transition
- Automotive (ASIL B): >97% stuck-at, >90% transition
- Automotive (ASIL D): >99% stuck-at, >95% transition, path delay coverage
- Aerospace/Military: >99.9% with multiple fault models

INDUSTRY: The automotive standard ISO 26262 and functional safety standard IEC 61508 specify minimum diagnostic coverage requirements that map directly to DFT fault coverage targets.

5.4 Test Patterns

5.4.1 Pattern Format

ATPG output patterns are typically stored in:

- WGL (Waveform Generation Language): Synopsys-native, full timing information
- STIL (Standard Test Interface Language): IEEE 1450 standard, ATE-portable
- ASCII/binary: Simple formats for simulation

5.4.2 Scan Test Sequence

Each scan test pattern consists of:

24. Scan-in phase: Shift N bits serially into scan chains (N clock cycles)
25. Capture phase: Apply capture clock (1–2 cycles), sample combinational logic
26. Scan-out phase: Shift N bits out while shifting in next pattern

```
Pattern count example:
1M gate design, full scan, 16 chains of 500 FFs each
Shift time = 500 cycles per pattern
10,000 patterns × 500 cycles = 5M cycles total
At 100 MHz: 50ms test time per chip (without compression)
```

5.5 Deterministic vs Random Testing

5.5.1 Deterministic ATPG

Deterministic ATPG generates specific patterns targeting specific faults. Every bit of the pattern is calculated to activate and propagate a specific fault. High fault coverage, but patterns can be large.

5.5.2 Pseudo-Random Testing

Pseudo-random patterns are generated by Linear Feedback Shift Registers (LFSR). These are simple to generate and good for random-pattern-resistant (RPR) faults. Used in BIST applications.

5.5.3 Hybrid Approach

Industry uses hybrid: ATPG for targeted coverage + random patterns for RPR faults + at-speed patterns for transition faults. Most production test programs include all three types.

5.6 Compression Techniques in ATPG

ATPG compression reduces pattern count while maintaining coverage:

- Pattern merging: Compatible patterns (same care bits) merged into one
- X-filling: Don't-care bits filled to improve compression ratio or power
- Broadcast patterns: Same data sent to multiple chains via compressor

```
# TetraMAX compression command example
set_atpg -compression on
run_atpg -capture_cycles 1 -merge full
```

SECTION SUMMARY

- ✓ ATPG algorithmically generates test patterns — replacing manual test vector creation
- ✓ Fault Coverage > 95% SAF is the minimum industry standard; automotive requires >99%
- ✓ Pattern formats: WGL (Synopsys), STIL (IEEE standard) for ATE compatibility
- ✓ Each scan pattern involves shift-in → capture → shift-out phases
- ✓ Deterministic ATPG + random patterns + at-speed patterns = complete test program
- ✓ SAT-based ATPG engines handle complex designs with high efficiency

SECTION 6: Built-In Self-Test (BIST)

6.1 Introduction to BIST

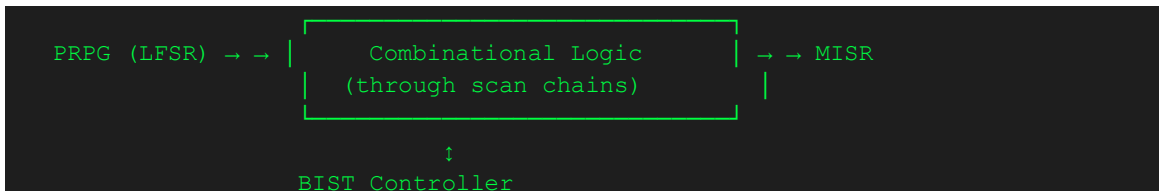
Built-In Self-Test (BIST) embeds test circuitry directly on the chip, enabling the chip to test itself without external ATE equipment. BIST is essential for:

- Field testing (chip tests itself during power-up)
- Periodic in-system testing (automotive fault detection)
- Memories (SRAM, ROM) that cannot be efficiently tested with scan
- Embedded analog blocks (ADC, PLL self-test)

6.2 Logic BIST (LBIST)

6.2.1 LBIST Architecture

LBIST uses on-chip hardware to generate test patterns and evaluate responses, eliminating the need for external ATE for logic testing:



- PRPG (Pseudo-Random Pattern Generator): LFSR generates stimulus
- Scan chains: Test patterns loaded into scan FFs, responses captured
- MISR (Multiple-Input Signature Register): Compresses scan out data into signature
- BIST Controller: Orchestrates shift/capture cycles, compares final signature

6.2.2 LBIST Operation

27. BIST Controller initializes PRPG and MISR to seed values
28. N shift/capture cycles are performed (typically 10,000–100,000)
29. Final MISR signature is compared to golden signature
30. Match = pass; Mismatch = fail (chip is defective)

6.2.3 LBIST Coverage

LBIST achieves 90–95% stuck-at coverage with sufficient pattern count. It is less efficient than deterministic ATPG for high coverage but requires no external test data.

INDUSTRY: Synopsys STAR Memory System and Cadence Modus MBIST are commercial LBIST/MBIST IP products used widely in industry. Custom BIST is rare for complex designs.

6.3 Memory BIST (MBIST)

6.3.1 Why Memories Need Special Treatment

SRAMs and ROMs have unique failure mechanisms not covered by logic fault models:

- Stuck-At Cell Faults: Memory cell stuck at 0 or 1

- Coupling Faults: One cell's write affects adjacent cells
- Transition Faults: Cell fails to switch state
- Address Decoder Faults: Wrong address accessed
- Sense Amplifier Faults: Read circuit failures
- Open/Short Word/Bit Lines: Wire defects in memory array

6.3.2 March Test Algorithms

March algorithms are the industry-standard MBIST approach. They apply specific read/write sequences that detect coupling faults:

```
March C- Algorithm (detects all SAF and coupling faults):
↑(w0) ↑(r0,w1) ↑(r1,w0) ↓(r0,w1) ↓(r1,w0) ↑(r0)
↑ = ascending address order
↓ = descending address order
r0 = read expecting 0, w0 = write 0
```

The March C- algorithm detects SAF, AF (Address Faults), CF (Coupling Faults), and transition faults in $O(N)$ operations for N memory cells.

6.3.3 MBIST Controller Architecture

An MBIST controller for each memory instance generates:

- Address sequence (ascending/descending as per march algorithm)
- Data pattern (0s, 1s, checkerboard, walking-1s)
- Read/write control signals
- Expected data comparison logic
- Pass/fail status register

INDUSTRY: Modern SoCs may have 100+ memory instances. Hierarchical MBIST with a top-level test controller that orchestrates individual memory BIST controllers is the standard approach.

6.4 PRPG – Pseudo-Random Pattern Generator

A PRPG generates pseudo-random bit sequences using a Linear Feedback Shift Register (LFSR). An N -bit LFSR generates a maximal-length sequence of $2^N - 1$ patterns before repeating.

```
Example: 4-bit LFSR with taps at positions 4 and 3:
Feedback = Q[4] XOR Q[3]
Sequence: 0001→0011→0111→1111→1110→1100→1000→...
Period:  $2^4 - 1 = 15$  patterns
```

6.5 MISR – Multiple-Input Signature Register

MISR compresses test response data into a compact signature. It is essentially a parallel LFSR where each stage XORs with one bit of the response data. The final signature (typically 32 or 64 bits) represents the compressed response.

```
MISR operation (4-bit example):
Each clock: MISR_next = LFSR_shift(MISR) XOR Response_data
After N cycles: compare MISR to golden signature
Aliasing probability  $\approx 2^{-(\text{bits})}$  per incorrect signature match
```

NOTE: For a 32-bit MISR, aliasing probability is $2^{-32} \approx 2.3 \times 10^{-10}$. This is acceptably low for production testing.

6.6 Advantages and Limitations of BIST

6.6.1 Advantages

- No external ATE required for field/in-system testing
- Fast test execution (on-chip clock vs. ATE serial interface)
- Parallel testing of identical memory instances
- Enables periodic self-test in safety-critical applications (ISO 26262)

6.6.2 Limitations

- Area overhead: BIST controller + PRPG + MISR add silicon area
- Cannot achieve 100% coverage for complex deterministic faults
- Aliasing risk: Very small probability of false pass
- Hard to diagnose failures (signature tells you fail, not what failed)
- LBIST power: High switching activity during LBIST can exceed thermal limits

INDUSTRY: For automotive ASIL-D, LBIST is mandatory for periodic self-test during system operation. The BIST runs in the background at reduced clock speed to manage power.

SECTION SUMMARY

- ✓ BIST embeds test circuitry on-chip — enables field testing without ATE
- ✓ LBIST uses PRPG+MISR to test logic through scan chains autonomously
- ✓ MBIST uses March algorithms for comprehensive SRAM/ROM testing
- ✓ MISR compresses response data into a compact signature for pass/fail decision
- ✓ BIST is mandatory in automotive (ISO 26262) for periodic in-system self-test
- ✓ BIST coverage (90-95%) is lower than deterministic ATPG — used complementarily

SECTION 7: Boundary Scan (JTAG) – IEEE 1149.1

7.1 IEEE 1149.1 Standard Overview

The IEEE 1149.1 standard, commonly called JTAG (Joint Test Action Group), defines a standardized interface for testing assembled PCBs and embedded chip cores. Originally developed for board-level testing, JTAG is now universally embedded in SoCs for:

- PCB interconnect testing (detecting solder joint defects)
- Chip-level debug and emulation
- FPGA and microcontroller programming
- Embedded core access (Coresight Debug Infrastructure)
- Hierarchical DFT access for multi-die SiP packages

7.2 JTAG Pin Interface

JTAG defines a mandatory 4-pin interface (plus optional TRST):

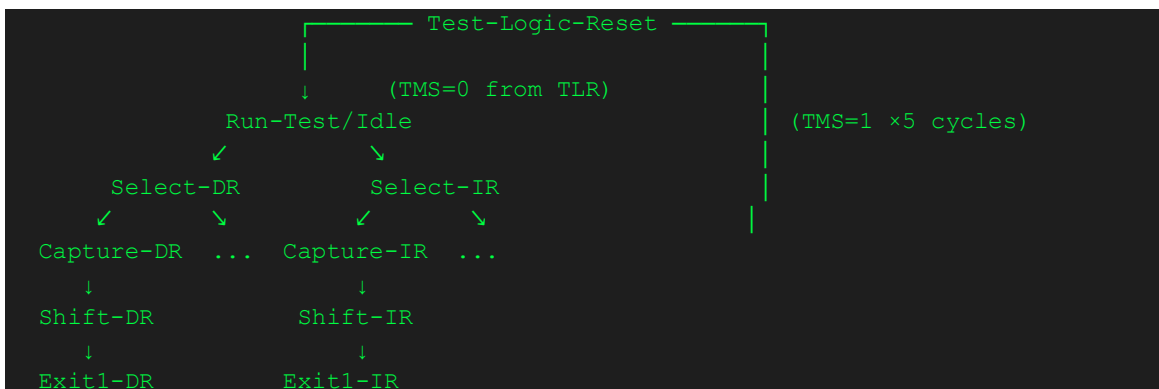
- TCK (Test Clock): Shifts data through the JTAG chain
- TMS (Test Mode Select): Controls TAP state machine transitions
- TDI (Test Data In): Serial data input to the chain
- TDO (Test Data Out): Serial data output from the chain
- TRST (Test Reset, optional): Asynchronous reset of TAP controller

NOTE: JTAG uses a fixed 4-pin interface regardless of chip complexity. This pin efficiency is a key advantage over parallel test interfaces.

7.3 TAP Controller (Test Access Port)

7.3.1 TAP State Machine

The TAP controller is a 16-state finite state machine driven by TMS and TCK. Key states:



- Test-Logic-Reset: Default safe state; all test logic inactive
- Run-Test/Idle: Idle state; used during BIST operation
- Capture-DR: Loads parallel data into selected data register
- Shift-DR: Serially shifts data register through TDI→TDO
- Update-DR: Transfers shift register to output register

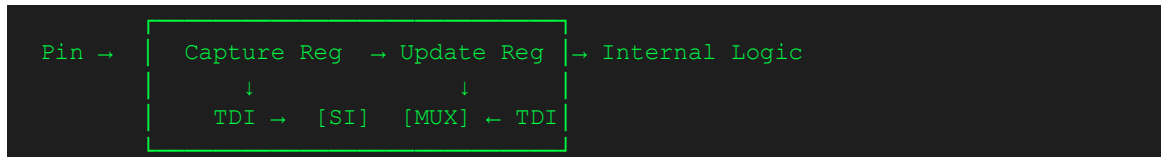
7.3.2 Instruction Register (IR)

The IR holds the current JTAG instruction. Typically 4–10 bits wide. IR is loaded through the IR scan path, which is separate from the data register path.

7.4 Boundary Scan Cells

7.4.1 Cell Structure

A boundary scan cell (BSC) is inserted at each I/O pin of the chip. It consists of a capture register, update register, and multiplexer:



In normal mode: I/O pin connects directly to internal logic (transparent).

In test mode: BSC can sample pin values (capture) or force pin values (override).

7.5 JTAG Instructions

7.5.1 Mandatory Instructions

- **BYPASS:** All data registers bypassed; TDI → 1-bit bypass register → TDO. Allows quick chain traversal
- **SAMPLE/PRELOAD:** Captures current I/O pin values into boundary scan registers without affecting chip operation
- **EXTEST:** External test mode. Forces output values from boundary registers, captures input values. Used to test PCB interconnects

7.5.2 Optional Instructions

- **INTEST:** Internal test. Drives values into chip's internal logic from boundary scan cells. Tests chip-level logic
- **RUNBIST:** Activates on-chip BIST while chip is in JTAG control
- **IDCODE:** Shifts out chip identification code (32-bit register with manufacturer, device, version info)
- **USERCODE:** User-defined 32-bit code (useful for firmware version identification)

7.5.3 EXTEST Example (PCB Test)

```
# EXTEST flow for PCB net connectivity test
1. Load EXTEST instruction into all chips on PCB
2. Drive known values on outputs of Driver chip
3. Capture resulting values at inputs of Receiver chip
4. Compare captured vs expected: mismatch = open/short on PCB trace
```

TIP: JTAG chain on a PCB connects multiple chips in series: TDO of chip N → TDI of chip N+1. A single TCK/TMS/TDI/TDO interface tests the entire board.

7.6 JTAG Use Cases in Modern SoC

- JTAG-based debug: ARM CoreSight uses JTAG/SWD for processor debug and trace

- Hierarchical DFT: JTAG used to access internal scan chains in multi-die packages (IEEE 1149.1-2013)
- Flash programming: Most microcontrollers use JTAG for factory and field programming
- Functional verification: RTL simulation uses JTAG to access internal registers
- Failure analysis: After silicon failure, JTAG used to read/write internal state for debug

SECTION SUMMARY

- ✓ JTAG (IEEE 1149.1) defines a 4-pin (TCK/TMS/TDI/TDO) standardized test interface
- ✓ TAP Controller is a 16-state FSM controlling all JTAG operations
- ✓ Boundary Scan Cells at each I/O pin enable pin-level control and observation
- ✓ EXTEST tests PCB interconnects; INTEST tests chip internal logic; BYPASS enables quick chain bypass
- ✓ JTAG is universal: debug, programming, DFT access, and hierarchical testing all use JTAG
- ✓ Modern SoCs embed multiple JTAG-accessible data registers for DFT, debug, and manufacturing

SECTION 8: Scan Compression

8.1 The Need for Compression

Without scan compression, large modern designs face an unacceptable test time problem. Consider a 100-million-FF design:

```
Without compression:
  100M FFs / 100 chains = 1M FFs per chain
  1M shift cycles × 10,000 patterns = 10 billion test cycles
  At 100 MHz = 100 seconds per chip – commercially unacceptable

With 128:1 compression:
  128,000 internal chains, 1000 FFs per chain
  1000 shift cycles × 10,000 patterns = 10 million cycles
  At 100 MHz = 0.78 seconds per chip – acceptable!
```

Scan compression reduces both test time and ATE data volume (test data stored in ATE memory), directly reducing test cost.

8.2 Decompressor Architecture

The decompressor expands K physical scan inputs (from ATE) into M internal scan chains. Common architectures:

8.2.1 Illinois Scan (XOR-based)

```
scan_in[0] ————┐ internal_chain[0]
scan_in[1] ————┤ internal_chain[1]
scan_in[0] XOR scan_in[1] — internal_chain[2]
...
```

XOR combinations of physical inputs generate M unique patterns. Simple, area-efficient.

8.2.2 LFSR-based Decompressor

An LFSR seeded by the physical scan inputs generates pseudo-random patterns for internal chains. Used in Synopsys DFTMAX and Mentor Tessent EDT (Embedded Deterministic Test).

8.2.3 Ring Generator

A ring generator is a variation where scan input pins connect to multiple taps of a ring-configured shift register. Higher compression ratios possible vs. simple XOR network.

INDUSTRY: Synopsys DFTMAX Ultra achieves 256:1 compression using advanced ring generator architectures. Cadence Modus EDT and Siemens Tessent EDT are equivalent products.

8.3 Compressor Architecture

The compressor combines M internal chain outputs into K physical scan outputs:

```
internal_chain_out[0] ————┐ scan_out[0]
internal_chain_out[1] ————┤ XOR ┐
internal_chain_out[2] ————┤ scan_out[1]
...
```

XOR-based compressors are simplest. The compressor must be designed to avoid X-masking — when unknown values (X) at chain outputs corrupt the compressed output and cause false failures.

8.3.1 X-Tolerance

Xs in scan response data (from uninitialized memory, tri-state buses, analog interfaces) can XOR to corrupt compressed output. Compression-aware ATPG uses X-filling techniques to control X propagation.

8.4 Test Data Volume Reduction

Scan compression reduces:

- Scan-in data: Fewer physical pins means fewer bits to store in ATE memory
- Scan-out data: Fewer comparison bits → smaller expected data file
- Test time: Fewer shift operations in parallel chains

Data volume formula:

```
Without compression:  $K_{full\_chains} \times L \times P$  bits  
With compression:  $K_{physical} \times L_{internal} \times P$  bits  
Reduction =  $K_{physical} / K_{full\_chains} = 1/Compression\ Ratio$ 
```

8.5 Trade-offs in Scan Compression

- Area overhead: Decompressor + compressor add 2–5% logic area
- Compression vs. coverage: Higher compression → more X-masking risk → potentially lower coverage
- Diagnosis: Compressed patterns are harder to diagnose (must decompress to identify failing chain)
- Pattern count: More patterns needed to achieve same coverage vs. uncompressed

TIP: Most DFT engineers target 64:1 to 128:1 compression as the sweet spot — significant test time reduction with manageable area overhead and coverage impact.

SECTION SUMMARY

- ✓ Scan compression reduces test time by 32:1 to 256:1 by using on-chip decompressor/compressor
- ✓ Decompressor expands K physical scan inputs to M internal chains ($M \gg K$)
- ✓ XOR-based and LFSR-based decompressors are most common
- ✓ X-masking is the main risk in compression — caused by unknown values corrupting XOR outputs
- ✓ 64:1 to 128:1 compression is the industry sweet spot for test time vs. area/coverage trade-off
- ✓ DFTMAX (Synopsys), Modus EDT (Cadence), Tessent EDT (Siemens) are commercial solutions

SECTION 9: Low Power DFT

9.1 Scan Power Issues

Scan testing is notoriously power-intensive. During scan shift, every flip-flop toggles potentially every cycle as pseudo-random data shifts through the chain. This causes:

- Peak switching activity 5–10x higher than normal operation
- Excessive instantaneous current (IR drop on power grid)
- Thermal stress (chip temperature spikes)
- Potential gate oxide damage from prolonged high-activity operation

9.1.1 Quantifying Scan Power

```
Scan shift toggle rate ≈ 50% (random data)
Functional toggle rate ≈ 5-15% (typical data patterns)
Scan power ≈ 3-10x functional power
```

A chip designed to operate at 1W functional power may draw 5–10W during scan shift — requiring thermal management and potentially violating absolute maximum ratings.

WARNING: ATE power supplies have current limits. Chips with high scan power must be tested at reduced clock frequency or with power-aware patterns to stay within ATE limits.

9.2 Clock Gating During Scan

9.2.1 Problem

Functional clock gating cells disable clock branches when the corresponding logic is idle. During scan mode, all clock gates must be enabled (transparent) to clock all scan FFs. Simply forcing all clock gates open during scan can cause even higher power than normal operation.

9.2.2 Solution: Scan-Mode Clock Gate Control

DFT tools insert dedicated test logic to control clock gates during scan:

31. During scan SHIFT: Only the shifted chain's clocks are active — gating disabled selectively
32. During scan CAPTURE: All clocks enabled for exactly one cycle — brief power spike
33. After capture: Clock gating re-enabled immediately

```
# Synopsys DFT Compiler - handle clock gating
set_dft_signal -view existing_dft \
  -type TestMode -port scan_mode
set_clock_gating_check -setup 0.5 -hold 0.1
```

9.3 Power-Aware ATPG

9.3.1 Static Compaction with Power Constraints

Modern ATPG tools support power-aware pattern generation that limits switching activity:

- Peak power constraint: Limit maximum number of transitions per capture cycle
- Average power constraint: Limit average switching across the entire test sequence

- Segment-based testing: Divide design into power segments, test one at a time

```
# TetraMAX power-aware ATPG
set_atpg -capture_cycles 1 \
        -max_capture_transitions 1000 \
        -dynamic_power_reduction on
```

9.3.2 Low-Power X-Filling

X-filling (setting don't-care bits) significantly impacts scan power:

- 0-fill: Fill X bits with 0 — reduces toggling from 1→X transitions
- Adjacent fill: Fill X bits to match adjacent bit — reduces transition count
- Weighted random fill: Statistical X-fill to hit a target toggle rate

TIP: Adjacent X-filling can reduce scan shift power by 30–50% with no fault coverage penalty — it's a free power optimization in ATPG.

9.4 Techniques to Reduce Switching

9.4.1 Scan Segmentation

Divide the design into power domains and test one domain at a time. While testing segment A, segment B is clock-gated. This reduces instantaneous power at the cost of longer total test time.

9.4.2 Reduced Toggle Rate Patterns

Special ATPG modes generate patterns with artificially low toggle rates:

- Minimum transition fill: X bits filled to minimize transitions vs. previous pattern
- Linear decompressor with low-power seeds: Decompressor generates low-activity patterns

9.4.3 Scan Enable Gating

Rather than shifting all chains simultaneously, shift subsets of chains in rotation. This reduces simultaneous switching by a factor equal to the number of shift segments.

9.4.4 At-Speed Capture Power

The capture phase (1–2 cycles at full speed) can cause the highest instantaneous power spike. Techniques:

- Limit care bits per capture: ATPG with maximum capture transitions constraint
- Slow-speed capture for some patterns: Capture at lower frequency where allowed
- Power-safe patterns: Patterns specifically designed to avoid high-toggling functional states

INDUSTRY: In advanced nodes (7nm, 5nm), IR drop during scan capture can cause timing failures not present in functional operation — a major sign-off concern requiring power-aware ATPG.

SECTION SUMMARY

- ✓ Scan power is 3–10x functional power — a critical constraint in modern test program development
- ✓ Clock gating cells must be handled specially in scan mode to prevent excessive power
- ✓ Power-aware ATPG limits toggle rate per capture cycle — available in all major ATPG tools

- ✓ X-filling (adjacent fill, 0-fill) reduces scan shift power 30–50% with no coverage impact
- ✓ Scan segmentation tests one power domain at a time — reduces peak power at cost of time
- ✓ IR drop during scan capture causes timing failures — requires power-grid-aware signoff

SECTION 10: DFT Constraints & Timing

10.1 Scan Mode vs Functional Mode

A design operates in two distinct modes, each with different timing requirements:

10.1.1 Functional Mode

Normal operation. All paths must meet setup and hold timing constraints at the functional clock frequency. Scan FFs behave as standard DFFs (SE=0, MUX selects D_func).

10.1.2 Scan Mode

Test operation. The design runs at a test clock frequency (often lower than functional). Scan paths (SI→FF→Q→SI of next FF) are the critical paths. Special timing rules apply:

- Scan shift timing: SI-to-Q path must meet setup/hold at test clock frequency
- Scan capture timing: The capture clock must be applied at functional frequency for transition/delay tests
- Scan enable timing: SE must be stable with respect to the capture clock

10.2 Timing Constraints for Scan

10.2.1 SDC Constraints for Scan

```
# Create test clock (typically slower than functional)
create_clock -name TEST_CLK -period 10.0 [get_ports scan_clk]

# Set scan paths as exceptions during functional analysis
set_false_path -through [get_pins */SE]

# Scan enable false path (SE is quasi-static during test)
set_false_path -from [get_ports scan_en]

# Define scan path for shift timing analysis
set_multicycle_path -setup 2 -from [get_pins */SI] \
    -to [get_pins */D] -through [get_pins */SE]
```

10.2.2 Scan Hold Violations

During scan shift, hold time violations can corrupt scan data. The SI→FF path can be very short if adjacent FFs are physically close. Buffer insertion in scan chains resolves hold violations.

WARNING: Scan hold violations are common after scan stitching. Physical implementation tools must re-run timing analysis after scan insertion and fix hold violations before tapeout.

10.3 Test Clocks

10.3.1 Types of Test Clocks

- Slow-speed scan clock: Used during scan shift; typically 10–100 MHz vs. functional GHz
- At-speed capture clock: Applied at full functional frequency for transition/delay testing

- OCC (On-Chip Clock Controller): On-chip circuit that generates capture pulses at functional speed from a slow ATE clock

10.3.2 OCC Architecture

The OCC is a critical DFT component for at-speed testing:

```
Slow ATE Clock → [OCC] → Scan Shift Clock
                  → At-Speed Capture Pulse
```

The OCC generates a precisely timed capture clock pulse at the functional frequency from the ATE's slower interface clock. This is essential because ATE equipment cannot directly generate GHz-frequency clocks with the required precision.

INDUSTRY: OCC design is one of the most complex DFT challenges. Incorrect OCC implementation leads to false timing failures or missed real failures during at-speed test.

10.4 False Paths in Scan

Several paths in scan-inserted designs are false paths that should not be analyzed:

- SE-driven paths: Timing from SE to FF D-input through MUX — SE is static during capture
- Scan chain paths during functional mode: SI→FF paths when SE=0 (scan data doesn't propagate)
- Cross-domain paths in scan: Paths between FFs in different scan chains

```
# Set false paths for scan MUX paths
set_false_path -through [get_pins *scanmux*/S]
set_false_path -through [get_pins *SDFF*/SE]
```

10.5 Multi-Cycle Paths in Scan Context

Some designs use multi-cycle paths (MCPs) for slow-moving data. In scan mode, MCP exceptions must be handled carefully:

34. During scan shift: MCP paths behave as single-cycle paths (test clock is slow)
35. During capture: MCP setup constraints must be maintained for correct operation

```
# Multi-cycle path — functional mode only
set_multicycle_path -setup 2 \
  -from [get_cells REG_A] -to [get_cells REG_B]
  -exceptions {not_in_scan_capture}
```

SECTION SUMMARY

- ✓ Scan mode and functional mode have different timing constraints — both must close timing
- ✓ SDC: set_false_path for SE-driven paths; separate test_clock for scan shift analysis
- ✓ OCC generates at-speed capture pulses from slow ATE clock — critical for transition testing
- ✓ Scan hold violations are common after stitching — must be fixed before tapeout
- ✓ False paths through scan MUX must be declared to prevent STA from analyzing non-functional paths

SECTION 11: Physical DFT Considerations

11.1 Scan Chain Routing

After scan stitching (logical), the physical routing of scan chains from one FF to the next is performed during the routing step of physical design. Scan chain routing has unique challenges:

- Long routes: If FFs are stitched in netlist order (not physical proximity), chains may traverse the entire chip
- Congestion: Long scan routes consume routing resources needed for functional signals
- Hold violations: Short routes between physically adjacent FFs cause hold time violations

TIP: Placement-aware scan stitching, where FFs are stitched based on physical proximity after placement, reduces average scan route length by 60–80% vs. netlist-order stitching.

11.2 Placement-Aware Scan Stitching

11.2.1 Flow

36. Complete initial placement (place FFs based on functional connectivity)
37. Extract FF positions from DEF file
38. Re-stitch scan chains: connect nearest neighbor FFs within same clock domain
39. Re-run routing with improved scan chain topology

11.2.2 Commands

```
# Cadence Innovus placement-aware scan reorder
setScanReorderMode -skipAtpg false
reorderScan

# Synopsys IC Compiler II
set_scan_path_optimization_options \
    -placement_aware true
optimize_scan_chains
```

11.3 Routing Congestion from Scan

Scan chains can contribute significant congestion, especially in:

- Memory-dense regions: SRAM surrounds have high functional routing demand
- Clock tree regions: Pre-placed buffers create routing blockages
- I/O pad rings: Scan ports near pad ring create long routes to internal FFs

11.3.1 Mitigation Strategies

- Segment scan chains by physical region — chains stay local
- Add scan access points (dedicated scan ports mid-chip) for large designs
- Use scan compression to reduce number of long routes to scan ports
- Reserve routing tracks for scan in congested areas during floorplanning

11.4 Scan Chain Impact on Timing Closure

Scan stitching modifies FF connectivity, which affects:

- Hold time: Short scan paths need buffer insertion (increases area and power)
- Setup time: Long scan chains with unbalanced lengths create shift timing violations
- Clock skew: Scan FFs require balanced clock delivery even at test frequency

11.4.1 Scan Timing Closure Flow

40. Post-scan-insertion STA: Run timing analysis with test_clock constraints
41. Fix scan hold violations: Insert delay buffers in scan paths
42. Verify scan shift timing: Ensure all SI→Q paths meet test clock requirements
43. Verify capture timing: At-speed capture must meet functional clock constraints

INDUSTRY: In advanced nodes, scan hold fixing can add 3–5% to total cell count. Factor this into area budget during floorplanning.

11.5 Reordering for Debug

Physical scan reordering creates a challenge for failure diagnosis: the reordered chain order differs from the logical netlist order. Diagnosis tools must map physical chain positions back to logical net names.

Most EDA tools maintain a scan chain map file (STIL or ASCII) that records the mapping between physical FF position in the chain and the logical FF instance name. This file is essential for:

- Failure diagnosis: Map failing bit position to failing FF instance
- ATPG correlation: Ensure patterns match the physically-stitched chain order
- Post-silicon debug: Identify failing logic based on scan chain position

SECTION SUMMARY

- ✓ Placement-aware scan stitching reduces route length 60–80% vs. netlist-order stitching
- ✓ Scan routing congestion must be considered during floorplanning — reserve routing tracks
- ✓ Scan hold violations after stitching require buffer insertion — add 3–5% area budget
- ✓ At-speed capture timing must close at functional frequency — use OCC for precise pulse generation
- ✓ Scan chain map file is essential for diagnosis — must be generated and preserved

SECTION 12: DFT Verification

12.1 DFT DRC Checks

DFT Design Rule Checks verify that the scan-inserted netlist meets all DFT requirements before ATPG. DRC violations must be resolved before pattern generation.

12.1.1 Critical DFT DRC Checks

- Clock mixing: FFs from different clock domains mixed in same chain
- Asynchronous reset/set: FFs with async controls that cannot be overridden in scan mode
- Uncontrolled clock gates: Clock gates that are not controlled by test logic
- Bi-directional ports: I/O ports that must be forced to input mode during scan
- Feedback loops: Combinational feedback through scan chains
- Multiple drivers: Scan chain output driven by multiple sources
- Scan enable path violations: SE signal not reaching all FFs

12.1.2 DRC Commands

```
# Synopsys DFT Compiler DRC
dft_drc -verbose
report_dft_drc -all_violations

# Cadence Modus DRC
check_dft_rules
report_scan_chain_info
```

12.2 Scan Chain Verification by Simulation

12.2.1 Scan Shift Test

A basic simulation verifies scan chain connectivity: shift a known pattern (e.g., 10101010...) through all chains and verify the same pattern appears at scan_out after N shift cycles.

```
# Simulation test bench snippet
// Initialize
scan_en = 1;
scan_in = {N_CHAINS{1'b1}};

// Shift in all-1s pattern
repeat(CHAIN_LENGTH) @(posedge scan_clk);

// Verify scan_out
if (scan_out != {N_CHAINS{1'b1}})
    $error("Scan chain broken!");
```

12.2.2 Capture Test Simulation

After scan shift, apply capture clock and verify that the design correctly captures combinational logic values. Compare simulation response to ATPG-expected response.

12.3 Formal DFT Verification

Formal tools can verify scan chain connectivity without simulation:

- Formal scan connectivity checker: Proves scan path from scan_in to scan_out without simulation
- Equivalence checking: Verifies scan-inserted netlist is functionally equivalent to pre-scan netlist in functional mode

```
# Synopsys Formality - equivalence check
read_sverilog -r pre_scan_netlist.v
read_sverilog -i post_scan_netlist.v
set_top_design r:/WORK/chip_top
verify
```

12.4 Coverage Validation

After ATPG, coverage reports must be reviewed to ensure targets are met:

12.4.1 Coverage Report Analysis

```
# TetraMAX coverage report
report_statistics
# Output includes:
# Stuck-at coverage: 98.34%
# Transition coverage: 92.18%
# Pattern count: 4,521
# Atpg untestable: 234 (redundant logic)
```

12.4.2 Low Coverage Investigation

44. Identify uncovered fault list: report_faults -uncovered
45. Check if faults are truly untestable (redundant logic) or ATPG limitations
46. Add test points for hard-to-test logic if coverage target not met
47. Re-run ATPG with extended options for residual faults

SECTION SUMMARY

- ✓ DFT DRC checks must pass before ATPG — clock mixing and async set/reset are critical violations
- ✓ Scan chain simulation verifies physical connectivity end-to-end before tape-out
- ✓ Formal equivalence check ensures scan insertion does not change functional behavior
- ✓ ATPG coverage report must show >95% SAF coverage — investigate low-coverage faults
- ✓ report_faults -uncovered identifies specific fault locations needing attention

SECTION 13: Test Coverage & Metrics

13.1 Fault Coverage Definition

$$\text{Fault Coverage (FC)} = \text{Detected Faults} / (\text{Total Faults} - \text{ATPG Untestable}) \times 100\%$$

The denominator excludes truly untestable faults (redundant logic), focusing coverage on what can actually be tested. Industry-standard coverage reporting uses this formula.

13.2 Test Coverage vs Defect Coverage

13.2.1 Test Coverage

Test coverage measures what fraction of modeled faults are detected. It is a simulation metric — computed from netlist analysis and ATPG.

13.2.2 Defect Coverage

Defect coverage measures what fraction of real physical defects are detected. It depends on:

- Fault model accuracy: How well the model represents real defects
- Test coverage: Higher test coverage → higher defect coverage
- Technology-specific defect mechanisms: Some defects not modeled by standard fault models

Defect coverage is always estimated from field return data and process characterization — it cannot be computed directly from simulation.

NOTE: Fault coverage of 99% does not guarantee defect coverage of 99%. Some defects (especially at advanced nodes) are not captured by stuck-at or transition fault models. IDDQ and path delay tests fill these gaps.

13.3 Coverage Targets by Application

Different applications demand different coverage levels based on failure risk and safety requirements:

13.3.1 Consumer Electronics

- Stuck-At: >95%
- Transition: >85% (if at-speed test available)
- DPPM target: <10 (10 defective parts per million shipped)

13.3.2 Automotive (ISO 26262)

- ASIL A/B: >97% stuck-at, >90% transition
- ASIL C/D: >99% stuck-at, >95% transition, path delay coverage of critical paths
- DPPM target: <1 (extremely low for safety-critical applications)

13.3.3 Aerospace/Defense

- Stuck-At: >99.9%
- Multiple fault models required (SAF + transition + path delay + IDDQ)
- Zero DPPM target — parts tested to destruction for sample validation

13.4 Yield Impact Analysis

13.4.1 The Fault Coverage-Yield Relationship

Using the Williams-Brown model:

$$\text{Defect Level (DL)} = 1 - (\text{Yield})^{(1-\text{FC})}$$

Example: If wafer yield = 90% and FC = 95%:

$$\begin{aligned}\text{DL} &= 1 - (0.90)^{(1-0.95)} = 1 - (0.90)^{0.05} \\ \text{DL} &= 1 - 0.9947 = 0.53\% = 5,300 \text{ DPPM}\end{aligned}$$

If FC improves to 99%:

$$\begin{aligned}\text{DL} &= 1 - (0.90)^{(1-0.99)} = 1 - (0.90)^{0.01} \\ \text{DL} &= 1 - 0.9989 = 0.11\% = 1,100 \text{ DPPM}\end{aligned}$$

This demonstrates why improving coverage from 95% to 99% provides 5x improvement in shipped quality.

13.5 Coverage Improvement Strategies

- Add test points: Insert observation points (test point FFs) for hard-to-observe nodes
- Restructure logic: Remove redundant gates that create untestable faults
- Partial scan: Add scan to previously excluded FFs in hard-to-cover regions
- Extended ATPG: Run ATPG with higher effort level for residual faults
- Functional patterns: Add functional test vectors to complement ATPG

SECTION SUMMARY

- ✓ Fault Coverage = Detected/(Total-Untestable) — excludes truly redundant faults from denominator
- ✓ Defect coverage is estimated from field data — not directly computable from simulation
- ✓ Automotive ASIL D requires >99% SAF and >95% transition coverage
- ✓ Williams-Brown model: $\text{DL} = 1 - \text{Yield}^{(1-\text{FC})}$ — coverage directly determines DPPM
- ✓ 95%→99% FC improvement provides ~5x improvement in shipped quality (DPPM)

SECTION 14: Debug & Failure Analysis

14.1 Silicon Debug Overview

When a chip fails during test, the DFT engineer must identify the root cause. The debug flow moves from test-level symptoms to physical defect identification:

48. Identify failing patterns and failing scan chain bits
49. Map failing scan positions to logic instances (diagnosis)
50. Identify suspect nets and failure mechanisms
51. Physical failure analysis (cross-section, SEM imaging)

14.2 Failing Pattern Analysis

14.2.1 ATE Failure Data

ATE generates a fail log for each failing die:

```
# Example ATE fail log
PATTERN 1245: PASS
PATTERN 1246: FAIL
  Chain 3, Bit 127: Expected 1, Got 0
  Chain 7, Bit 89: Expected 0, Got 1
PATTERN 1247: FAIL
  Chain 3, Bit 127: Expected 1, Got 0
```

14.2.2 Interpreting Fail Logs

Consistent failures at the same chain/bit position across multiple patterns strongly suggest a physical defect at that FF or its driving logic. Random failures suggest marginal timing or power issues.

TIP: An 'SA0' signature (always reads 0 regardless of what's written) in the fail log is a strong indicator of an actual stuck-at-0 defect at that node.

14.3 Diagnosis Flow

14.3.1 Logic Diagnosis

Logic diagnosis tools (Synopsys TetraMAX Diagnosis, Cadence Modus Diagnosis) correlate the ATE fail data with the fault model to identify suspect fault locations:

```
# TetraMAX diagnosis
read_verilog post_scan_netlist.v
read_patterns failing_patterns.stil
read_failures ate_faillog.txt
run_diagnosis -effort high
report_diagnosis > diagnosis_report.txt
```

Output: list of suspect nets ranked by likelihood, with fault type (SA0/SA1/STR/STF) and confidence score.

14.3.2 Physical Failure Analysis (PFA)

Using diagnosis output, the physical failure analysis team:

52. Identifies the suspect location on the chip layout (GDS coordinates)
53. Performs passive voltage contrast (PVC) or electron beam probing to isolate defect
54. Cross-sections the suspect location using FIB (Focused Ion Beam)
55. Examines cross-section under SEM (Scanning Electron Microscope)
56. Identifies defect type: open via, metal bridge, gate oxide pinhole, etc.

14.4 Root Cause Identification

14.4.1 Systematic vs. Random Defects

- Systematic defects: Appear at the same location on every wafer — caused by process/lithography issue. Fix requires process change.
- Random defects: Appear randomly — caused by particles, contamination. Fix requires improved fab cleanliness.

14.4.2 Yield Learning Loop

DFT enables a critical yield learning feedback loop:

57. Diagnosis identifies suspect locations across many failing dice
58. Yield analysis tools map suspects to physical layout
59. Hot spots (regions with high defect density) identified
60. Process engineers investigate hot spots and implement fixes
61. Next wafer lot shows improved yield — feedback loop converges

INDUSTRY: Without DFT-enabled diagnosis, yield learning is nearly impossible. The yield improvement from systematic DFT diagnosis can accelerate time to volume production by months.

14.5 Common Debug Scenarios

14.5.1 Scan Chain Continuity Failure

Symptom: All bits in a chain shift incorrect values. Cause: Break in scan chain route (open via, missing buffer). Fix: Physical inspection at break point, ECO patch.

14.5.2 Hold Time Violation in Scan

Symptom: Random failures on bits near chain segment boundaries. Cause: Insufficient hold margin on scan paths after placement. Fix: Insert hold buffers in scan path.

14.5.3 Clock Mixing Corruption

Symptom: Systematic failures on specific chains only at certain pattern groups. Cause: Different-frequency clocks sharing a chain. Fix: Re-stitch chains by clock domain.

SECTION SUMMARY

- ✓ Silicon debug maps ATE fail patterns to physical die locations through logic diagnosis
- ✓ TetraMAX/Modus Diagnosis tools correlate fail logs with fault models to identify suspects
- ✓ Consistent same-position failures indicate physical defects; random failures indicate marginality
- ✓ PFA (cross-section + SEM) physically confirms the defect type after logic diagnosis

✓ DFT-enabled yield learning identifies systematic defects and accelerates process fixes

SECTION 15: Industry DFT Flow

15.1 Complete RTL to Silicon Test Flow

The complete DFT flow spans from RTL specification to silicon testing. Each phase has specific DFT tasks:

15.1.1 Phase 1: RTL & Specification

```
RTL Design → DFT Specification
DFT Engineer activities:
• Define DFT architecture (scan chains, compression, BIST)
• Specify DFT pins in package pin list
• Review RTL for DFT compliance (clock structures, async resets)
• Create DFT plan document
```

15.1.2 Phase 2: DFT-Aware Synthesis

```
RTL → [Logic Synthesis] → Gate-level Netlist (with scan)
DFT Engineer activities:
• Run DFT DRC on synthesized netlist
• Insert scan flip-flops (DFT Compiler / Modus)
• Insert MBIST controllers for memories
• Insert OCC for at-speed testing
• Verify scan insertion by simulation
```

15.1.3 Phase 3: ATPG

```
Scan-inserted Netlist → [ATPG] → Test Patterns
DFT Engineer activities:
• Run stuck-at ATPG (TetraMAX / Modus ATPG)
• Run transition ATPG with OCC constraints
• Achieve coverage targets
• Convert patterns to ATE format (STIL/WGL)
• Simulate patterns on gate-level netlist
```

15.1.4 Phase 4: Physical Design Integration

```
Netlist → [Floorplan → Place → CTS → Route] → Final Layout
DFT Engineer activities:
• Provide scan chain map to physical design team
• Verify physical scan stitching after placement
• Fix scan hold violations post-routing
• Run post-layout ATPG for final coverage verification
```

15.1.5 Phase 5: Tapeout Verification

```
Final Layout → [DFT Signoff] → GDSII Submission
DFT Engineer activities:
• Final DFT DRC clean
• Pattern simulation with post-layout timing (SDF)
• Coverage report signoff
• ATE program delivery
```

15.1.6 Phase 6: Silicon Testing

```
Fabricated Wafers → [Wafer Test] → [Package] → [Final Test]
Activities:
• Wafer-level (probe) test with ATE
• Pattern debugging if yield is low
• Diagnosis on failing dice
• Coverage correlation to actual yield
```

15.2 EDA Tools Overview

15.2.1 Synopsys

- DFT Compiler: Scan insertion, chain stitching, compression
- TetraMAX / TetraMAX Ultra: ATPG, fault simulation, diagnosis
- DFTMAX: Scan compression engine (ring generator based)
- IC Compiler II / Fusion Compiler: Physical design with DFT-aware scan reorder

15.2.2 Cadence

- Modus: Unified DFT platform (scan insertion + ATPG + compression + BIST)
- Genus: DFT-aware synthesis
- Innovus: Physical implementation with scan reorder capability

15.2.3 Siemens (Mentor)

- Tessent Shell: Comprehensive DFT platform
- Tessent EDT: Embedded Deterministic Test (compression)
- Tessent MBIST: Memory BIST insertion
- Tessent Diagnosis: Failure analysis tool

15.3 Real Project Flow Example (SCL 180nm)

```
Project: 32-bit RISC processor, SCL 180nm, full scan

Step 1: RTL analysis
dft_drc -verbose -check_clocks -check_resets
Result: 3 async reset violations found → fixed in RTL

Step 2: Scan insertion
set_scan_configuration -chain_count 4 \
                      -style multiplexed_flip_flop
insert_dft
Result: 2,847 scan FFs in 4 chains of ~712 FFs each

Step 3: ATPG
run_atpg -stuck_at
Result: 98.7% SA coverage, 4,123 patterns

Step 4: Verification
simulate_patterns → 100% patterns pass gate-level sim
```

INDUSTRY: In the C2S program at SCL 180nm, this flow was executed end-to-end including seal ring integration and full DFT signoff — demonstrating the complete industry flow.

SECTION SUMMARY

- ✓ DFT spans 6 phases: Spec → Synthesis → ATPG → Physical → Tapeout → Silicon Test
- ✓ DFT Compiler (Synopsys), Modus (Cadence), Tessent (Siemens) are the main platforms
- ✓ DFT engineer is active in every phase — not just scan insertion
- ✓ Post-layout ATPG with SDF timing is required for final coverage signoff
- ✓ ATE program delivery (STIL/WGL patterns) is the DFT engineer's final deliverable

SECTION 16: Common Mistakes & Best Practices

16.1 Poor Scan Planning

16.1.1 Common Mistake: Late DFT Planning

Starting DFT insertion after synthesis is complete forces rework. Clock domain issues, missing test ports, and package pin conflicts discovered late cause expensive schedule delays.

TIP: BEST PRACTICE: DFT planning must start at RTL design phase. DFT pin allocation in the package must happen before floorplanning.

16.1.2 Common Mistake: Ignoring Clock Domains

Mixing clocks in scan chains is a critical DFT violation. Teams that don't analyze clock domain structure before scan insertion face extensive rework.

Best Practice: Create a clock domain map early. Ensure scan stitching tool is configured with correct clock domain assignments.

16.1.3 Common Mistake: Insufficient Test Pins

Under-allocating test pins forces either low chain count (long test time) or no scan compression (even longer test time). Negotiate adequate test pin count in package specification.

16.2 Low Coverage Issues

16.2.1 Cause: Untestable Logic

Redundant logic (multiple identical paths, constant outputs) creates untestable faults. While not defects, they inflate the untestable fault count and can hide real coverage problems.

Best Practice: Remove redundant logic during RTL cleanup. Use synthesis optimization to eliminate constants.

16.2.2 Cause: Hard-to-Control Nodes

Nodes deep in the logic cone with many fan-in levels are hard to control from scan FFs. ATPG may leave these as 'possibly detected'.

Best Practice: Use SCOAP analysis to identify hard-to-test nodes early. Add test points (observation scan FFs) for nodes with SCOAP > threshold.

16.2.3 Cause: Async Set/Reset Violations

FFs with async set or reset that cannot be overridden by scan prevent scan from loading known values. These FFs cannot be tested properly.

Best Practice: Convert async resets to synchronous where possible. If async is required, add scan override logic.

16.3 Power Issues During Test

16.3.1 Thermal Failures During Test

High scan switching activity causes thermal spikes. If the chip operates at its maximum power limit during test, thermal shutdown or permanent damage can occur.

Best Practice: Use power-aware ATPG. Test at reduced clock frequency if needed. Add on-chip temperature monitoring.

16.3.2 IR Drop in Scan Mode

Power grid IR drop during scan capture can cause functional failures unrelated to defects — false failures that reduce yield.

Best Practice: Run power grid analysis with scan capture patterns. Ensure IR drop in scan mode meets power grid specifications.

16.4 Debug Challenges

16.4.1 Missing Scan Chain Map

After physical reordering, if the updated scan chain map is not regenerated, diagnosis tools produce incorrect results. Always regenerate the scan chain database after any physical reordering.

16.4.2 Pattern Mismatch

Patterns generated for pre-layout netlist may not correctly test the post-layout netlist if ECOs were applied. Always re-run ATPG or at minimum re-simulate patterns after significant ECOs.

16.5 Industry Best Practices Summary

62. Start DFT planning at RTL specification — not post-synthesis
63. Always run DFT DRC and resolve all violations before ATPG
64. Target >98% SAF coverage for standard designs; >99% for automotive
65. Use scan compression (64:1 minimum) for any design with >100K FFs
66. Implement OCC for designs requiring at-speed (transition) testing
67. Run power-aware ATPG to prevent thermal/IR drop test failures
68. Regenerate scan chain map after every physical reorder
69. Keep scan pattern simulation (gate-level) in the tape-out sign-off checklist
70. Document all DFT exclusions with technical justification
71. Plan BIST for all embedded SRAM instances — never skip memory testing

SECTION SUMMARY

- ✓ DFT planning must start at RTL phase — late DFT insertion causes expensive rework
- ✓ Clock mixing in scan chains is the #1 DFT violation — always configure clock domain separation
- ✓ Async reset FFs require special handling — convert to synchronous where possible
- ✓ Power-aware ATPG is mandatory for advanced nodes — thermal/IR drop failures are real
- ✓ Scan chain map must be regenerated after physical reorder — critical for diagnosis accuracy
- ✓ >98% SAF coverage is the production minimum — automotive requires >99%

SECTION 17: Complete DFT Example – 4-bit ALU

17.1 Design Description

We will walk through a complete DFT implementation of a simple 4-bit ALU design with registers. This illustrates the end-to-end DFT flow in a self-contained example.

```
// Pre-DFT RTL
module alu_top (
    input      clk, rst_n,
    input  [3:0] a, b,
    input  [1:0] op,
    output [3:0] result
);
    reg [3:0] reg_a, reg_b, reg_out;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) {reg_a, reg_b, reg_out} <= 0;
        else begin
            reg_a <= a; reg_b <= b;
            case(op)
                2'b00: reg_out <= reg_a + reg_b;
                2'b01: reg_out <= reg_a - reg_b;
                2'b10: reg_out <= reg_a & reg_b;
                2'b11: reg_out <= reg_a | reg_b;
            endcase
        end
    end
    assign result = reg_out;
endmodule
```

17.2 DFT Analysis

17.2.1 Flip-Flops Identified

3 registers × 4 bits = 12 flip-flops total: reg_a[3:0], reg_b[3:0], reg_out[3:0]

17.2.2 DFT Issues

- rst_n is asynchronous active-low reset — DFT DRC violation
- No scan ports defined in the interface

17.2.3 RTL Fix for DFT

```
// DFT-friendly RTL (synchronous reset)
always @(posedge clk) begin
    if (!rst_n || test_mode)
        {reg_a, reg_b, reg_out} <= 0;
    else begin ... end
end
```

17.3 Post-Synthesis DFT Insertion

```
# DFT Compiler script for alu_top
```

```

read_verilog alu_top_synth.v
current_design alu_top

create_clock -name clk -period 5.0 [get_ports clk]

# Add DFT ports
set_dft_signal -view spec -type ScanEnable \
               -port scan_en -active_state 1
set_dft_signal -view spec -type ScanIn \
               -port scan_in
set_dft_signal -view spec -type ScanOut \
               -port scan_out

set_scan_configuration -chain_count 1 \
                       -style multiplexed_flip_flop
dft_drc
insert_dft
write_verilog alu_top_dft.v

```

17.4 Post-Insertion Netlist

```

// After DFT insertion: 12 FFs become 12 scan FFs
// One scan chain: scan_in → [FF0]→...→[FF11] → scan_out

// Scan chain sequence (after stitching):
// scan_in → reg_a[0] → reg_a[1] → reg_a[2] → reg_a[3]
//           → reg_b[0] → reg_b[1] → reg_b[2] → reg_b[3]
//           → reg_out[0] → reg_out[1] → reg_out[2] → reg_out[3]
//           → scan_out

```

17.5 ATPG Run

```

# TetraMAX ATPG for alu_top
read_verilog alu_top_dft.v
read_sdf alu_top.sdf ;# for timing info

set_atpg -stuck_at on
run_atpg

# Expected output:
# Total faults:      248
# Detected faults:   244
# ATPG untestable:    3 (redundant paths in adder)
# Stuck-at coverage: 98.4%
# Pattern count:      47

```

17.6 Scan Test Waveform

Cycle	scan_en	clk	scan_in	scan_out	Operation
1	1	↑	1	X	Shift-in bit 0
2	1	↑	0	X	Shift-in bit 1

...		...		...		...		...		...
12		1		↑		1		X		Shift-in bit 11
13		0		↑		X		X		Capture cycle
14		1		↑		1		1		Shift-out bit 0
...		...		...		...		...		Shift-out...
25		1		↑		0		0		Shift-out bit 11

SECTION SUMMARY

- ✓ End-to-end DFT on a 12-FF design: async reset fixed, scan FFs inserted, ATPG run
- ✓ Chain of 12 scan FFs requires 12 shift cycles per test pattern
- ✓ 98.4% stuck-at coverage achieved with 47 patterns — typical for simple combinational logic
- ✓ 3 untestable faults from adder redundancy — acceptable and documented
- ✓ This same flow scales to designs with millions of FFs using compression and parallel chains

SECTION 18: DFT Interview Questions & Answers

18.1 Basic Level Questions

Q1: What is DFT and why is it needed?

Answer: DFT (Design for Testability) is a set of design techniques that add test logic to ICs to make manufacturing testing feasible. It is needed because complex chips have internal nodes inaccessible from external pins, and manufacturing defects must be detected before shipping. Without DFT, achieving >90% fault coverage on a complex design is practically impossible.

Q2: What is the difference between a fault and a defect?

Answer: A defect is a physical imperfection in the manufactured chip (e.g., broken metal line). A fault is an abstract model of the defect's effect on circuit behavior (e.g., stuck-at-0 on that wire). We test for faults to detect defects. One physical defect can cause multiple logical faults.

Q3: What is a stuck-at fault?

Answer: A stuck-at fault assumes a signal line is permanently held at logic 0 (SA0) or logic 1 (SA1) regardless of inputs. Physical cause: open circuits often cause SA0, shorts to power cause SA1. Stuck-at fault coverage is the baseline DFT metric.

Q4: What are the pins of a JTAG interface?

Answer: TCK (Test Clock), TMS (Test Mode Select), TDI (Test Data In), TDO (Test Data Out), and optional TRST (Test Reset). These 4–5 pins provide access to all JTAG-controlled test registers regardless of chip complexity.

Q5: What is scan chain length and why does it matter?

Answer: Scan chain length is the number of scan flip-flops in a single chain. It determines test time (shift cycles = chain length per pattern). Longer chains = longer test time. Industry optimum: 500–3000 FFs per chain, balanced across multiple parallel chains.

18.2 Intermediate Level Questions

Q6: What is scan compression and what are typical compression ratios?

Answer: Scan compression uses on-chip decompressor and compressor logic to expand a small number of physical scan I/O pins into many internal scan chains. A decompressor (XOR network or LFSR) expands K physical inputs into M internal chains. Typical compression ratios: 64:1 to 128:1 for balanced test time reduction vs. area overhead.

Q7: What is an OCC and why is it needed?

Answer: An On-Chip Clock Controller (OCC) generates at-speed capture pulses from a slow ATE clock. It is needed for transition fault testing because ATE equipment cannot directly provide GHz-frequency clocks with the required precision. The OCC synthesizes a fast capture pulse internally, triggered by the slow ATE clock.

Q8: What is fault coverage and how does it relate to defect level?

Answer: Fault coverage (FC) = detected faults / total testable faults. Defect level $DL = 1 - \text{Yield}^{(1-FC)}$. Higher FC exponentially reduces DL. Going from 95% to 99% FC provides approximately 5x improvement in shipped quality (DPPM).

Q9: How does MBIST differ from logic scan testing?

Answer: MBIST uses dedicated march algorithms to test SRAM-specific fault mechanisms (coupling faults, address decoder faults, sense amplifier failures) that scan-based logic tests cannot detect. MBIST controllers are embedded on-chip and run autonomously, testing each memory array with sequential read/write patterns.

Q10: What is clock mixing and why is it a DFT violation?

Answer: Clock mixing occurs when flip-flops from different clock domains are stitched into the same scan chain. During scan shift, all clocks run simultaneously. If clocks have different phases or are asynchronous, data shifts out of sequence causing X-corruption. Fix: configure scan stitching to separate chains by clock domain.

18.3 Advanced Level Questions

Q11: Explain Launch-on-Shift vs Launch-on-Capture for transition fault testing.

Answer: Both are at-speed ATPG methodologies. Launch-on-Shift (LoS): the last shift clock pulse launches the transition at full speed; capture is one functional cycle later. Simpler to implement but affected by shift clock quality. Launch-on-Capture (LoC): two functional capture pulses are applied; first launches the transition, second captures. More timing-accurate (both pulses from functional clock) but harder to implement — requires OCC support for two consecutive at-speed pulses.

Q12: What is X-masking in scan compression and how is it addressed?

Answer: X-masking occurs when unknown (X) values at internal chain outputs propagate through the XOR compressor and corrupt the compressed output, causing false failures. X sources: uninitialized memories, tri-state buses, analog interfaces. Solutions: X-filling techniques (fill don't-care ATPG bits to cancel X values at compressor inputs), X-tolerant compressor design (use specific XOR topologies where X values cancel), blocking X sources with initialization sequences.

Q13: How would you improve stuck-at coverage from 94% to 99%?

Answer: (1) Run ATPG with higher effort: `set_atpg -effort high`. (2) Identify uncovered faults: `report_faults -uncovered`. (3) Check if uncovered faults are ATPG_untestable (redundant logic) — remove redundant logic. (4) Add test points: insert observation scan FFs for high-SCOAP nodes. (5) Check DFT DRC violations that might be blocking pattern propagation. (6) Verify clock gating cells are correctly handled in scan mode.

Q14: A chip passes all stuck-at ATPG patterns in simulation but fails in silicon at speed. What are the likely causes?

Answer: (1) Transition/delay faults: static ATPG doesn't test timing. Need at-speed patterns with transition ATPG. (2) IR drop during capture: power grid droops at functional frequency causing timing failures. Run power-aware ATPG and power grid analysis. (3) OCV/process variation: timing margins insufficient at worst-case corner. Check STA sign-off corners. (4) Electromigration or NBTI: reliability-induced degradation not present at first test. (5) At-speed clock not correctly reaching all FFs: OCC or clock distribution issue.

18.4 Scenario-Based Questions

Q15: You have a design with 50% memory (SRAM) and 50% logic. How do you plan the DFT architecture?

Answer: Memory (SRAM): implement MBIST for each SRAM instance using march C- algorithm. Use hierarchical MBIST controller. Logic: full scan with scan compression. Estimate chain count from FF count and test time budget. Add OCC for transition fault testing of critical timing paths. JTAG interface to control both scan and MBIST through a unified test access mechanism. Estimate power: scan shift power must stay within ATE current limits.

Q16: Your ATPG shows 20% of faults are ATPG_untestable. The design manager is concerned. How do you respond?

Answer: ATPG_untestable means the tool PROVED these faults are logically equivalent to fault-free behavior — they are REDUNDANT faults. They cannot cause any observable difference even if a real defect creates them. This is acceptable and means the redundant logic is providing no functional value. Recommendation: investigate why redundant logic exists (design issue?) and consider removing it to improve area. The remaining 80% testable faults achieve the targeted coverage — the 20% untestable do not reduce quality.

SECTION SUMMARY

- ✓ DFT interviews test both theoretical knowledge and practical implementation experience
- ✓ Always explain fault vs defect, coverage vs defect level, and ATPG vs BIST distinctions
- ✓ OCC, scan compression, and at-speed testing are the advanced topics most asked
- ✓ Scenario questions test your ability to make architectural decisions with trade-offs
- ✓ Know the complete flow: RTL → scan insertion → ATPG → physical → silicon test

END OF DOCUMENT

Design for Testability – Complete Practical & Theoretical Guide

Authored by Banothu Bapuji | M.Tech VLSI System Design
CVR College of Engineering, Hyderabad

ABOUT THE BOOK

This book provides a comprehensive understanding of Design for Testability (DFT) techniques used in modern ASIC and VLSI design. It covers everything from fundamental fault models and scan design to advanced topics such as ATPG, scan compression, BIST, and silicon debug. With a strong focus on practical implementation and industry methodologies, this guide enables engineers to design highly testable and reliable chips while achieving high fault coverage yield.

KEY FEATURES

- Complete coverage of DFT from basics to advanced
- Detailed explanation of scan design and ATPG
- Real-world industry flow and methodologies
- Coverage of LBIST, MBIST, and JTAG
- Practical debugging and failure analysis techniques
- Focus on test coverage, yield, and silicon reliability

ABOUT THE AUTHOR

Banothu Bapuji (M.Tech VLSI System Design)

Banothu Bapuji is a VLSI enthusiast specializing in ASIC design, timing analysis, and DFT methodologies. This book reflects a strong focus on practical understanding and industry-ready skills, making complex concepts accessible to students and engineers.

